IoT-Based Smart Energy Meter with Telegram Alerts

Real-Time Household Power Monitoring via MQTT Dashboard, I^2C LCD Display, and Automated Telegram Notifications

Author	Abdul Rahman
Module	Personal Project
Institution	University of Moratuwa
Date	Jan 2026

Contents

1. Project Overview

This project implements a complete IoT-based smart energy meter capable of monitoring household electrical consumption in real time, without the need for heavy hardware or a complex installation. The system measures six key electrical parameters — voltage, current, active power, cumulative energy, frequency, and power factor — and makes them available simultaneously on a local I^2C LCD display, an MQTT-driven web dashboard, and a paired Telegram bot.

The core sensing element is the PZEM-004T v4.0 module paired with an external current transformer (CT), interfaced to a DOIT ESP32 DevKit V1 microcontroller over a dedicated hardware serial port. The ESP32 handles WiFi connectivity, MQTT publishing to the public HiveMQ broker, and HTTPS-based Telegram alerts in a single firmware loop.

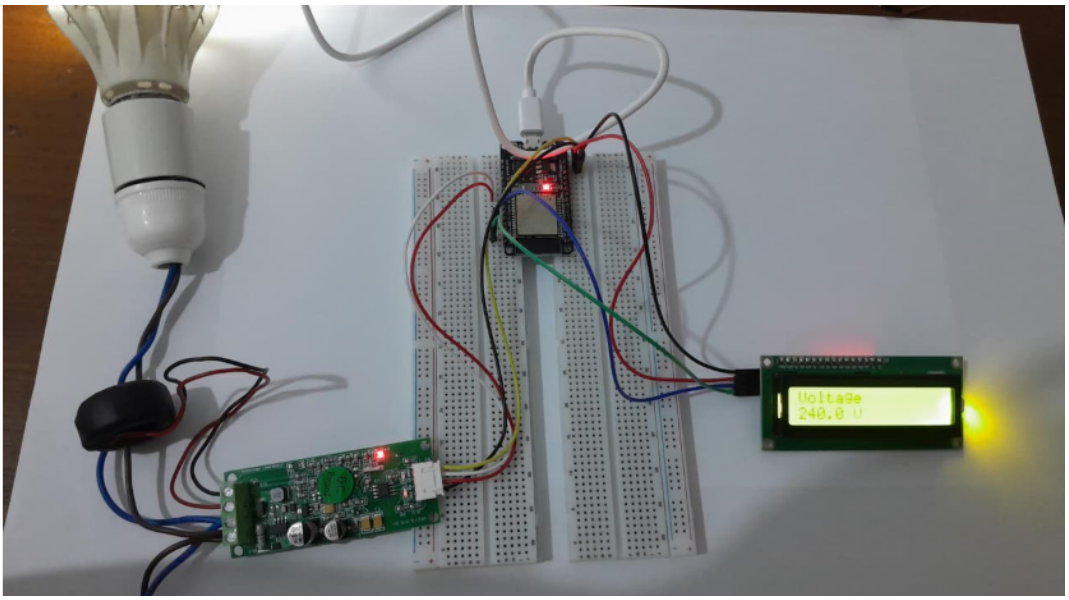

Figure 1. Overview of the completed IoT Smart Energy Meter setup.

The project goal was to produce a low-cost, remotely accessible energy monitor that alerts the user when abnormal conditions — specifically over-voltage — are detected, while continuing to publish live data and update the LCD uninterrupted.

2. System Architecture

2.1 Block Diagram

The system is divided into four functional layers: sensing, processing, communication, and display. The PZEM-004T module acquires raw electrical measurements from the mains supply via the external CT clamp. These readings are streamed to the ESP32 over a UART interface. The ESP32 processes the data and simultaneously drives three output channels — the I^2C LCD, the MQTT broker, and the Telegram notification service.

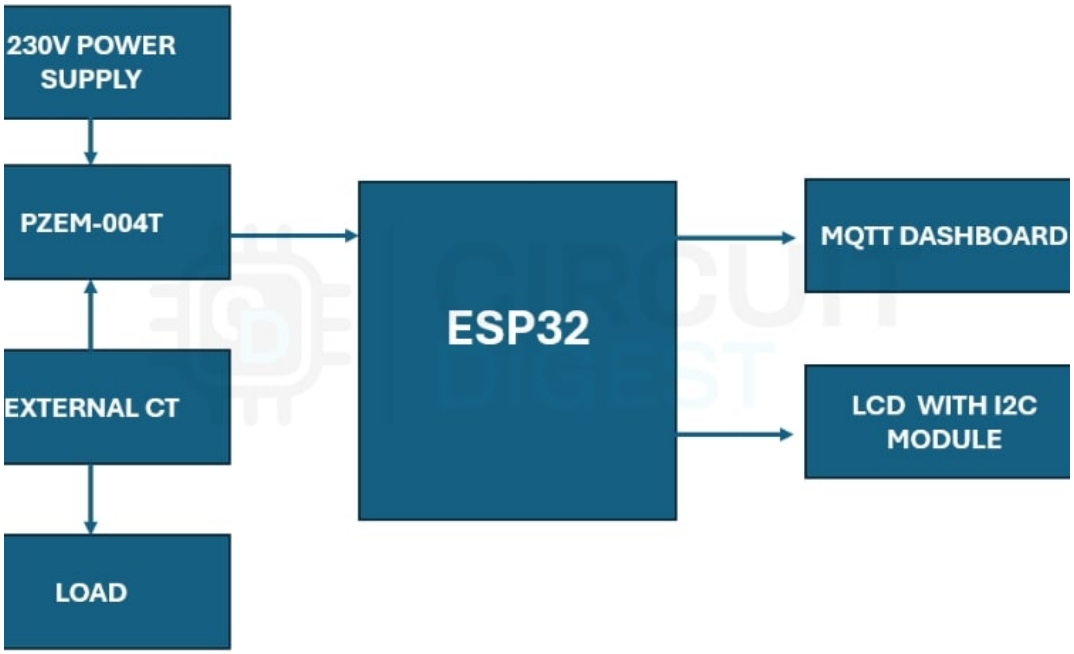

Figure 2. High-level block diagram showing the four-layer system architecture.

2.2 Circuit Diagram

The ESP32 communicates with the PZEM-004T module over Serial2, with GPIO 17 as `RX` and GPIO 16 as `TX`. The I^{2}C LCD module connects to GPIO 21 (SDA) and GPIO 22 (SCL) at address `0x27`. The PZEM module is connected to the mains loop through the external CT clamp, which clamps around the live conductor without requiring direct mains wiring to the microcontroller.

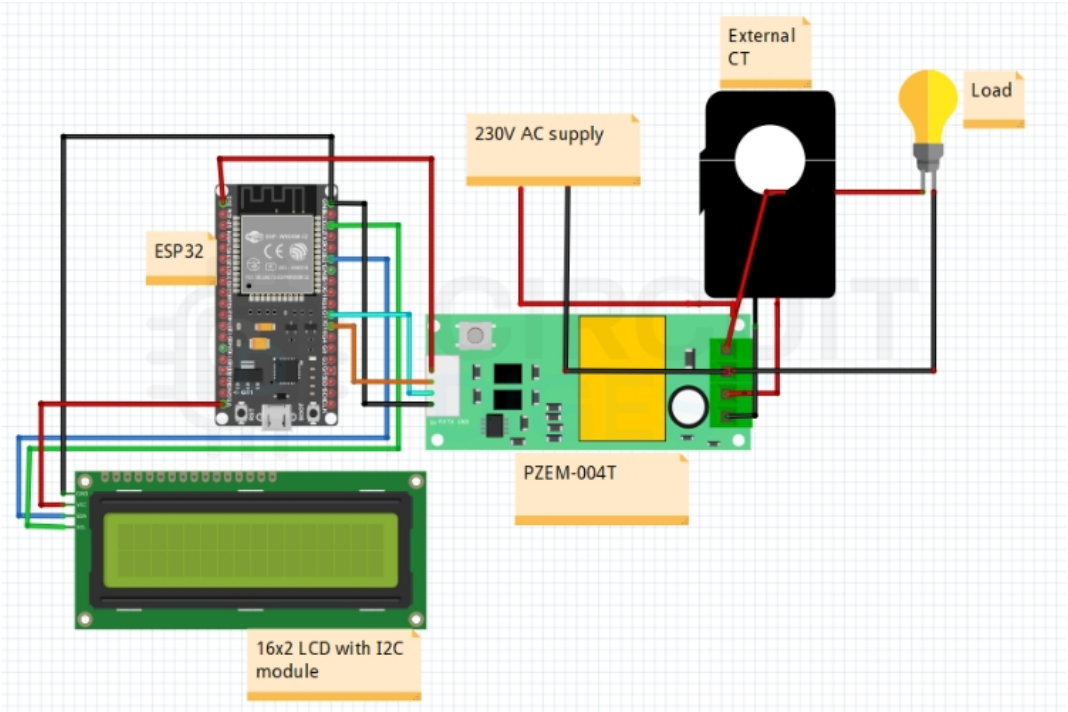

Figure 3. Full circuit diagram showing ESP32, PZEM-004T, and LCD connections.

3. Hardware Components

3.1 Component List and Cost

All components were sourced locally in Sri Lanka. The total bill of materials came to Rs. 5,360, making this a low-cost solution suitable for domestic deployment.

Component	Qty	Price (Rs.)
ESP32 DOIT DevKit V1 (MD0245)	1	1,390
I²C Module	1	150
LCD 16×2	1	470
PZEM-004T v4.0 + External CT	1	2,450
Breadboard	2	420
Jumper Wires	—	70
4-pin JST (for PZEM)	—	60
USB-C Data Cable	1	350
Total		**5,360**

3.2 PZEM-004T Module

The PZEM-004T v4.0 is the sensing backbone of the system. It measures AC voltage, current, active power, energy accumulation, line frequency, and power factor over a single UART interface. The external CT clamp eliminates the need to break into the mains circuit, making installation safer and reversible.

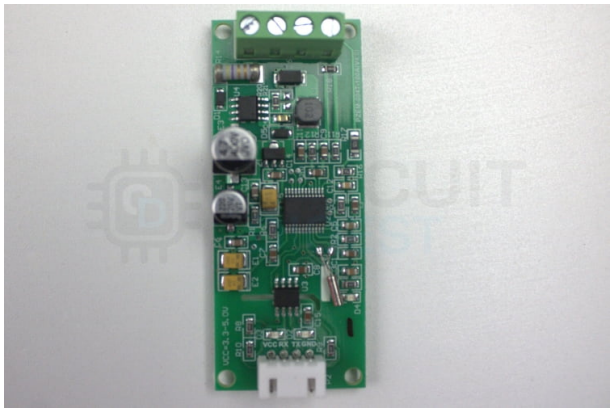

Figure 4. PZEM-004T v4.0 module used for AC electrical parameter measurement.

Parameter	Specification
Voltage range	80–260 V AC
Current range	0–100 A (with external CT)
Power range	0–23 kW
Energy range	0–9999.99 kWh
Frequency range	45–65 Hz
Interface	UART (9600 baud, 8N1)

Figure 5. PZEM-004T module connected with the external clamp-type CT sensor.

4. Firmware Design

4.1 Library Initialisation

The firmware depends on six Arduino libraries. The PZEM004Tv30 library abstracts the serial protocol for the sensor module. WiFi and PubSubClient handle network connectivity and MQTT publishing. HTTPClient and WiFiClientSecure are used for HTTPS calls to the SMS and Telegram APIs.

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <PZEM004Tv30.h>
#include <WiFi.h>
#include <PubSubClient.h>
#include <WiFiClientSecure.h>
#include <HTTPClient.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);
HardwareSerial PZEMSerial(2);
PZEM004Tv30 pzem(PZEMSerial, 17, 16); // RX=17, TX=16
```

4.2 Reading Electrical Parameters

All six electrical parameters are read from the PZEM module in a single pass at the start of each loop iteration. If the voltage reading is below 10 V (indicating the mains supply is off or disconnected), the remaining values are forced to zero to prevent publishing spurious data.

```
float V  = pzem.voltage();
float I  = pzem.current();
float P  = pzem.power();
float E  = pzem.energy();
float F  = pzem.frequency();
float PF = pzem.pf();

// Guard against mains-off condition
if (isnan(V) || V < 10.0) {
    I = 0.0;  P = 0.0;
    E = 0.0;  F = 0.0;  PF = 0.00;
}
```

4.3 MQTT Publishing

Each parameter is published to a dedicated MQTT topic on the public HiveMQ broker at `broker.hivemq.com:1883`. A composite JSON payload is also published to a single aggregated topic, which the web dashboard subscribes to for real-time updates.

```
const char* tV    = "unique/energy/voltage";
const char* tI    = "unique/energy/current";
const char* tP    = "unique/energy/power";
const char* tE    = "unique/energy/energy";
const char* tF    = "unique/energy/frequency";
const char* tPF   = "unique/energy/pf";
const char* tJSON = "unique/energy/data";

client.publish(tV,  String(V,  1).c_str(), true);
client.publish(tI,  String(I,  3).c_str(), true);
// ... (repeat for P, E, F, PF)

String json = "{";
json += "\"V\":"  + String(V,  1) + ",";
json += "\"I\":"  + String(I,  3) + ",";
json += "\"P\":"  + String(P,  1) + ",";
json += "\"E\":"  + String(E,  3) + ",";
json += "\"F\":"  + String(F,  1) + ",";
json += "\"PF\":" + String(PF, 2);
json += "}";
client.publish(tJSON, json.c_str(), true);
```

4.4 LCD Display Rotation

The 16×2 LCD cycles through all six parameters in sequence, displaying each for two seconds before clearing and advancing to the next screen. This provides a self-contained local readout without any external device.

```
lcd.clear(); lcd.setCursor(0,0);
lcd.print("Volt:");  lcd.print(V, 1);  lcd.print("V");
delay(2000);
lcd.clear(); lcd.setCursor(0,0);
lcd.print("Curr:");  lcd.print(I, 3);  lcd.print("A");
delay(2000);
// ... (Power, Energy, Frequency, Power Factor)
```

4.5 High-Voltage Alert via SMS

When the measured voltage exceeds 255 V, the firmware triggers an HTTPS POST request to the CircuitDigest Cloud SMS API. A one-minute cooldown timer (`smsCooldown`) prevents alert flooding during sustained over-voltage events.

```
if (V > 255) {
    if (millis() - lastSMS > smsCooldown) {
        sendSMS();
        lastSMS = millis();
    }
}

void sendSMS() {
    WiFiClientSecure secureClient;
    secureClient.setInsecure();
    HTTPClient http;
    String apiUrl = "https://www.circuitdigest.cloud/send_sms?ID="
                    + String(templateID);
    http.begin(secureClient, apiUrl);
    http.addHeader("Authorization", apiKey);
    http.addHeader("Content-Type", "application/json");
    String payload = "{\"mobiles\":\"" + String(mobileNumber)
                   + "\",\"var1\":\"Energy meter\""
                   + ",\"var2\":\"High Voltage\"}";
    http.POST(payload);
    http.end();
}
```

5. Telegram Bot Integration

5.1 Overview

The Telegram bot, named *VoltSentry*, provides a secure, two-way command interface over HTTPS. The bot uses a pairing-code mechanism to ensure only the authorised user can receive alerts or query device status. All bot credentials and the paired chat ID are stored in the ESP32's non-volatile storage (NVS) using the Preferences library, so the pairing survives power cycles.

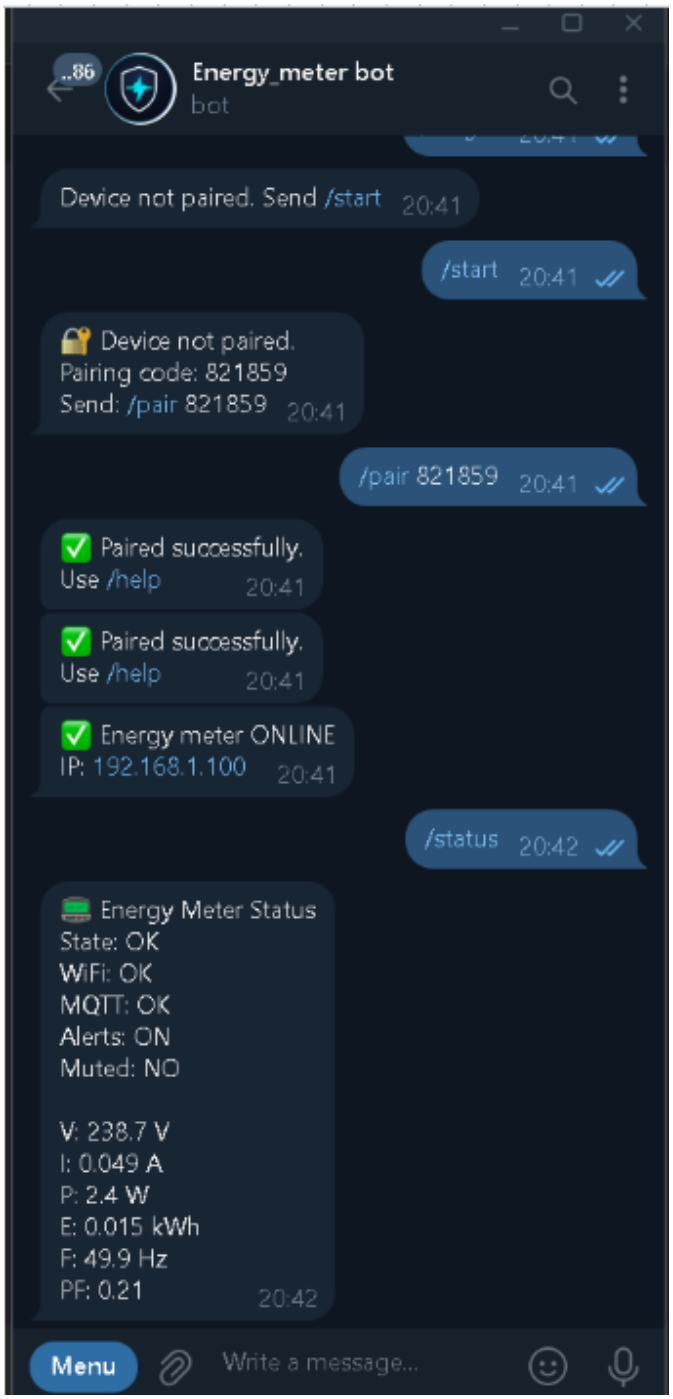

Figure 6. Telegram alert message received on the user's phone when high voltage is detected.

5.2 Command Set

Command	Function
`/start`	Initiates device pairing using a one-time code
`/status`	Returns a formatted snapshot of all six live readings
`/alerts on|off`	Enables or disables Telegram alert notifications
`/mute` *duration*	Suppresses alerts for a specified period (e.g. `10m`, `2h`, `1d`)
`/unmute`	Cancels an active mute
`/forget`	Clears the pairing and resets the device to unpaired state
`/help`	Lists all available commands

5.3 Polling Architecture

The bot polls for new Telegram updates every few seconds via `bot.getUpdates()`. On first boot after a power cycle, a priming pass drains any stale updates to avoid replaying old commands. Subsequent polls request only updates with an ID greater than the last processed update, which is persisted to NVS between reboots.

```
void telegramPoll() {
    if (WiFi.status() != WL_CONNECTED) return;
    if (millis() - lastTgPollMs < TG_POLL_MS) return;
    lastTgPollMs = millis();

    if (!tgPrimed) {              // drain stale updates on first
   boot
        int n = bot.getUpdates(0);
        if (n > 0) tgLastUpdateId = updToInt(
                       bot.messages[n-1].update_id);
        prefs.putInt("last_uid", tgLastUpdateId);
        tgPrimed = true;
        return;
    }
    int n = bot.getUpdates(tgLastUpdateId + 1);
    for (int i = 0; i < n; i++) {
        handleTelegramCommand(
            bot.messages[i].chat_id,
            bot.messages[i].text);
    }
    prefs.putInt("last_uid", tgLastUpdateId);
}
```

6. Web Dashboard

6.1 Design and Technology

The web dashboard is a single-page HTML application that connects directly to the HiveMQ public broker via the MQTT-over-WebSocket protocol, using the open-source mqtt.js library. No server-side backend is required; the dashboard can be opened from any device with a browser on the same network or from any public internet host since HiveMQ is a public broker.

The visual design uses a dark-mode aesthetic with a deep navy-to-dark-blue gradient background and blue accent colours for live metric cards. Six parameter cards display voltage, current, power, energy, frequency, and power factor in real time. A Chart.js time-series graph provides a rolling history of the selected parameter.

Figure 7. MQTT web dashboard showing live energy readings from the ESP32.

7. Software Architecture Summary

The main firmware loop runs four concurrent tasks interleaved by non-blocking timers, avoiding the use of an RTOS. Each task has an independent interval:

Task	Interval	Function
PZEM read	2 s	Samples all six electrical parameters
MQTT publish	2 s	Publishes individual topics and JSON payload
LCD update	2 s	Advances to next parameter screen
Telegram poll	3 s	Checks for new bot commands
Alert check	5 s	Evaluates over-voltage and sends notification
WiFi / MQTT guard	Every loop	Reconnects if connection is lost

The setup routine initialises the I^2C bus, LCD, PZEM UART, NVS Preferences, WiFi, MQTT client, and Telegram TLS client in sequence before entering the main loop. A simulation mode (`simMode`) is supported via serial commands, allowing the firmware to be tested without a live mains connection.

8. Discussion and Limitations

The system successfully demonstrates simultaneous real-time monitoring across three output channels with a total hardware cost under Rs. 5,500. Several design decisions and limitations are worth noting.

The HiveMQ public broker is used without authentication, meaning any client that knows the topic strings can read the published data. For a production deployment, a private broker with TLS client certificates would be appropriate. Similarly, the Telegram bot token and WiFi credentials are stored in plaintext in the firmware; these should be moved to a provisioned NVS namespace or served via a captive-portal configuration interface.

The over-voltage alert threshold of 255 V is hard-coded. A more flexible implementation would allow the user to configure thresholds via a Telegram command (e.g. `/setalert V 260`) and persist them to NVS. Under-current detection (load disconnection) and over-energy accumulation alerts would also be practical extensions.

The dashboard HTML file must be served from a local or hosted web server for WebSocket connections to work correctly in all browsers; opening it directly from the filesystem (`file://`) may be blocked by some browser security policies.

The GIF animation of the LCD cycling cannot be embedded in a static PDF; the `animate` package can be used if the source frames are available as individual images for an interactive PDF target.

9. Conclusion

This project demonstrates a complete, low-cost IoT energy monitoring pipeline built around an ESP32 microcontroller and the PZEM-004T AC measurement module. The system measures six electrical parameters in real time, displays them on a local I^2C LCD, publishes them to an MQTT broker for live web-dashboard consumption, and delivers automated Telegram alerts when abnormal conditions are detected — all within a single cooperative firmware loop.

The Telegram bot integration goes beyond simple one-way alerting: the pairing mechanism, mutable alert states, mute timers, and NVS persistence make it a practical remote control interface. The total component cost of approximately Rs. 5,360 positions this as an accessible solution for domestic energy awareness, with a clear upgrade path to private broker infrastructure, configurable thresholds, and multi-load monitoring.